DIP Session#1

CSE 4246:
Digital Image Processing Sessional

Dr. S.M. Jahangir Alam (JA)
Professor, Dept. of CSE
&
Md. Maskur Al Shal Sabil (MAS)

Contents

DIP Sessional#01:

Module#1.1: Introduction to Python, OpenCV, and Image I/O.
Module#1.2: Read, Display, and Save Images
Module#1.3: Color Space Conversion (RGB, BGR, Gray, HSV, LAB, YCrCb)
Module#1.4: Image Sampling and Quantization
Module#1.5: Image conversion to digital pixels

DIP Sessional#01:
Module#1.1: Introduction to Python, OpenCV, and Image I/O.
Module#1.2: Read, Display, and Save Images
Module#1.3: Color Space Conversion (RGB, BGR, Gray, HSV, LAB, YCrCb)
Module#1.4: Image Sampling and Quantization
Module#1.5: Image conversion to digital pixels

DIP Sessional#02:

Module#2.1: Image Enhancement: Image Contrast, Image Histogram, CLAHE, Histogram Equalization Matching, Brightness, Contrast, Gamma, Negative
Module#2.2: Image Transformation: Crop, Resize, Rotate, Flip

DIP Sessional#03:

Module#3.1: Image Filtering: Mean, Median, Gaussian, Bilateral
Module#3.2: Edge Detection: Sobel, Prewitt, Roberts, Canny operators
Module#3.3: Image Segmentation: Thresholding, Global, Adaptive, Otsu thresholding, Otsu, Watershed, Region Growing, K-Means

DIP Sessional#04:

Module#4.1: Morphological Operations: Erosion, Dilation, Opening, Closing, Skeletonization
Module#4.2: Frequency Domain Processing: DFT, FFT, Low-pass and High-pass filtering
Module#4.3: Image Compression: JPEG concept, DCT implementation

DIP Sessional#05:

Module#5.1: Feature Extraction: Harris, HOG, LBP, GLCM, SIFT, ORB, FAST
Module#5.2: Image Generation:
Module#5.3: Object Detection and Recognition: Contour detection, Template matching, YOLO, Face Detection
Module#5.4: Mini Project: Real-world image processing application

DIP Sessional#06:

Module#6.1: OCR & Computer Vision: EasyOCR, Tesseract, Image Classification

Module#1.1: Introduction to Python, OpenCV, and Image I/O.

Step 1: Install Anaconda

Download and install Anaconda from:

After installation, open **Anaconda Navigator** or **Anaconda Prompt**.

conda create -n dip python=3.12

Activate it:

conda activate dip

Step 2: Install Required Libraries

conda install numpy matplotlib jupyter

conda install -c conda-forge opencv

conda install -c conda-forge scikit-image

conda install pillow

Or install everything with pip:

pip install opencv-python scikit-image matplotlib pillow numpy

Module#1.1: Introduction to Python, OpenCV, and Image I/O.

Open **Anaconda Prompt**:
conda create -n dipcv python=3.12
conda install opencv matplotlib numpy

or,
pip install opencv-python matplotlib numpy

or
spyder

Activate it:
conda activate dipcv

Module#1.1: Introduction to Python, OpenCV, and Image I/O.

Step 1: Install Essential Packages

```
conda install -c conda-forge \
numpy scipy matplotlib pandas jupyter jupyterlab \
opencv scikit-image pillow imageio \
scikit-learn seaborn tqdm
```

Or install one by one:

```
conda install numpy
conda install scipy
conda install matplotlib
conda install pandas
conda install jupyter
conda install -c conda-forge opencv
conda install -c conda-forge scikit-image
conda install pillow
conda install imageio
conda install scikit-learn
conda install seaborn
```

Module#1.1: Introduction to Python, OpenCV, and Image I/O.

Step 4: Verify the Installation

```
import cv2
import numpy as np
import matplotlib
import skimage
import PIL import sklearn
import torch
import tensorflow as tf

print("OpenCV:", cv2.__version__)
print("NumPy:", np.__version__)
print("Matplotlib:", plt.matplotlib.__version__)
print("Pandas:", pd.__version__)
print("PyTorch:", torch.__version__)
print("TensorFlow:", tf.__version__)
print("All packages installed successfully!")
```

Libraries You'll Use Most

Library	Purpose
NumPy	Matrix and pixel operations
OpenCV (cv2)	Image processing and computer vision
Matplotlib	Display images and plots
Pillow (PIL)	Read and save images
scikit-image	Image processing algorithms
Jupyter Notebook	Interactive coding and visualization

Python

```
import numpy as np
import pandas as pd
```

linear algebra
numpy=numerical/array processing library

data processing, CSV file I/O (e.g. pd.read_csv)

NumPy→image/pixel computation
Pandas→table/data analysis

```
import os
```

OS (Operating System)
=file and folder management

```
print(os.path.join(dirname, filename))
```

used to work with **files, folders, and file paths**.

combine `os`, OpenCV, NumPy, and Matplotlib

Essential Packages & Uses

Package	Purpose
`opencv`	Image processing and computer vision
`numpy`	Matrix and pixel operations
`matplotlib`	Display images and plots
`scikit-image`	Image processing algorithms
`Pillow`	Read, write, and manipulate images
`scipy`	Scientific image operations
`pandas`	Dataset handling
`scikit-learn`	Machine learning
`jupyter`	Interactive notebooks
`ultralytics`	YOLO object detection
`torch, torchvision`	Deep learning and vision models
`tensorflow`	Deep learning
`easyocr`	Text recognition from images
`mediapipe`	Face, hand, and pose detection
`albumentations`	Image augmentation

Python

import cv2	# OpenCV
import matplotlib.pyplot as plt	# Image display
from PIL import Image	# Pillow
from skimage import io	# scikit-image I/O

from skimage import io, color, filters, morphology, exposure

```
print("OpenCV:", cv2.__version__)
print("NumPy:", np.__version__)
print("Imports successful!")
```

OpenCV (Open Source Computer Vision Library) standard Python interface

```
OpenCV: 4.13.0
NumPy: 2.0.2
Imports successful!
```

Python

import cv2	# OpenCV
import matplotlib.pyplot **as** plt	# Image display
from PIL **import** Image	# Pillow
from skimage **import** io	# scikit-image I/O

OpenCV (Open Source Computer Vision Library) standard Python interface

In DIP used for **reading, converting, resizing, displaying, and processing images**.

Python

import cv2	# OpenCV
import matplotlib.pyplot **as** plt	# Image display

matplotlib.pyplot=visualization and plotting
and
plt=short name for pyplot

This imports **Matplotlib's `pyplot` module** and gives it the short name **`plt`**.

DIP lab, **`plt`** is mainly used to **display images, pixel values, histograms, CDF graphs, and other plots**.

`cmap="gray"` → displays it in grayscale

Useful OpenCV functions for your DIP lab

Function	Purpose
`cv2.imread()`	Read image
`cv2.imwrite()`	Save image
`cv2.imshow()`	Display image
`cv2.cvtColor()`	Color conversion
`cv2.resize()`	Resize/sample image
`cv2.threshold()`	Image thresholding
`cv2.equalizeHist()`	Histogram equalization
`cv2.GaussianBlur()`	Gaussian smoothing
`cv2.medianBlur()`	Median filtering
`cv2.Canny()`	Edge detection

Module#1.2: Read, Display, and Save Images

```
import cv2
import matplotlib.pyplot as plt

Img_path=r"D:\DIPLab\Image\Fruits.jpg";

        img = cv2.imread(image_path)

if img is None:
    print("Image not found!")
else:
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    plt.imshow(img)
    plt.axis("off")
    plt.show()
```

Image Read and Display

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)

plt.axis("off")

plt.show()
```

Image loaded successfully.
Shape: (1021, 718, 3)

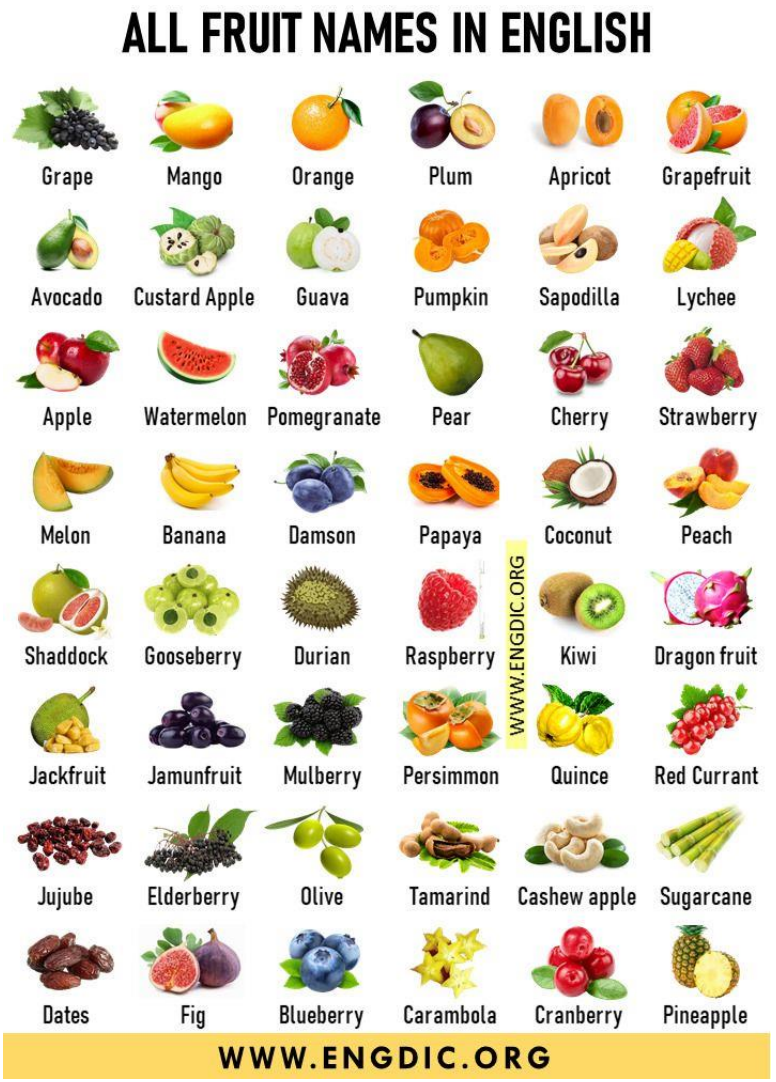


Image Read and Display

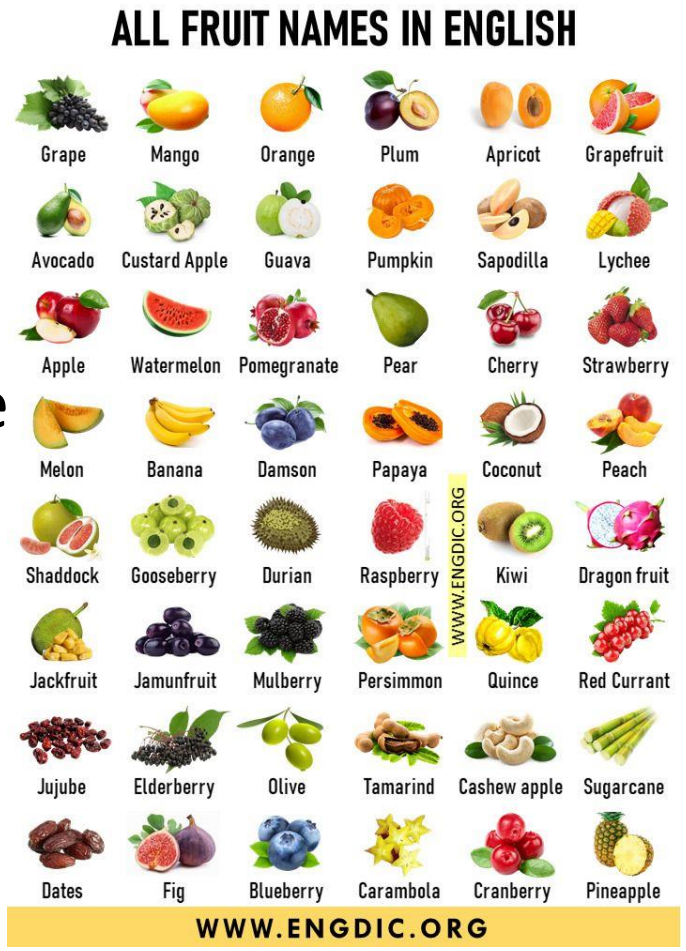
plt.figure() → Create figure

plt.imshow() → Put image in figure

plt.text() → Put pixel values on image

plt.title() → Add title

plt.show() → Display everything



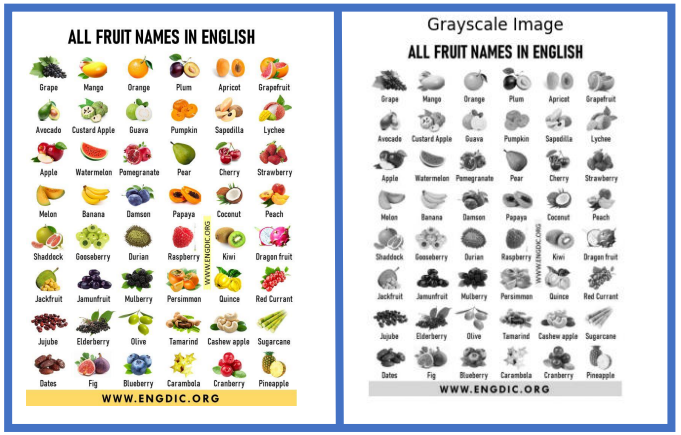
RGB Image to Grey Image

```
gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

```
plt.imshow(gray, cmap='gray')
plt.axis("off")
plt.title("Grayscale Image")
plt.show()
```




```
Plt.subplot(1,2,1)
plt.imshow(img_rgb)
Plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
```



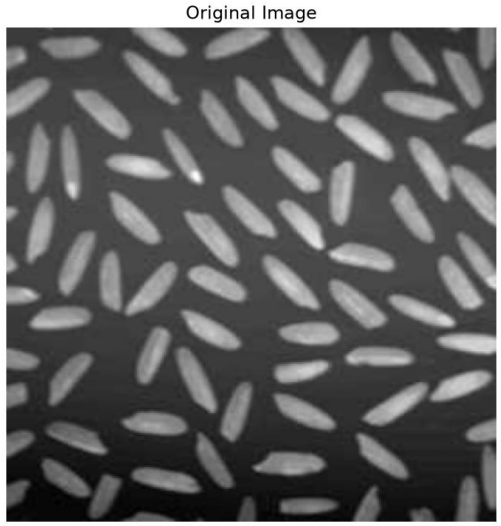
Save Images in a folder

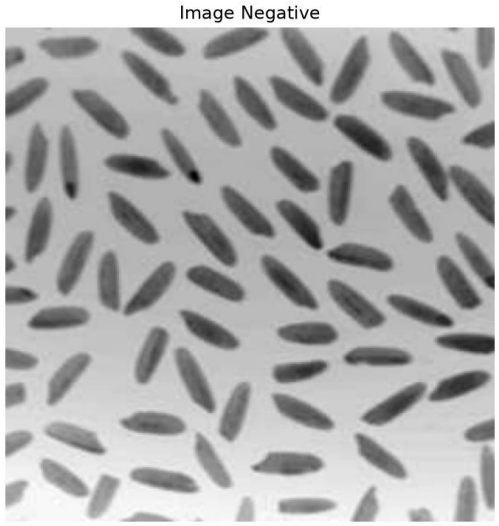
```
output_folder = r"D:\DIPLab\Results"
os.makedirs(output_folder, exist_ok=True)

cv2.imwrite(os.path.join(output_folder, "Fruits2.jpg"), img)

print("==================================")
print("Image Size :", img.shape)
print("Images saved successfully.")
print("Location :", output_folder)
print("==================================")
```

Image Negative





$$Image = \begin{bmatrix} 100 & 120 & 130 \\ 90 & 110 & 140 \\ 80 & 100 & 120 \end{bmatrix}$$

$$g(x,y) = L - 1 - f(x,y) = 255 - f(x,y)$$

```
img = cv2.imread(r"D:/DIPLab/Image/rice.tiff",
                 cv2.IMREAD_GRAYSCALE)
```

Image Negative

negative = 255 - img

- Mean, variance, and standard deviation
- Binary Image
- Gaussian Noise in Image
- Image conversion to Digital Image
- Image Sampling and Quantization
- Pixel Connectivity Graph
- Image Interpolation using KNN
- Color Conversion

Estimate Mean and Variance

- Img_path=r"D:/DIPLab/Image/Multicolor-MagnificFlower.bmp";
- img = cv2.imread(Img_path)

Estimate parameters

mean = np.mean(img)

variance = np.var(img)

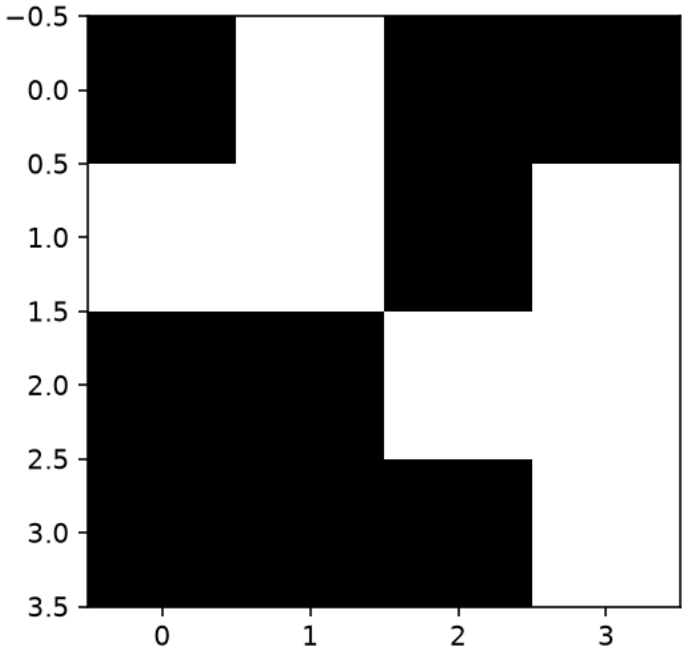
std = np.std(img)

3×3 Pixel Values:
[156 171 189]
[91 94 93]
[129 63 63]

Create a Binary Image

```
#Create a Binary Image
import numpy as np
import matplotlib.pyplot as plt
img = np.array([
        [0,1,0,0],
        [1,1,0,1],
        [0,0,1,1],
        [0,0,0,1]
], dtype=np.uint8)

plt.imshow(img, cmap='gray')
plt.show()
```



Gaussian Noise in Image

```
# Read grayscale image
Img_path=r"D:/DIPLab/Image/rice.tiff";
img = cv2.imread(Img_path,cv2.IMREAD_GRAYSCALE)
```

Gaussian noise values *N* (mean = 0, std dev = 2)

Assume, noise values (drawn from $\mathcal{N}(0, 2^2)$):

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

0.52	-1.37	2.14
-0.48	1.05	-2.31
0.76	-0.12	1.83

Gaussian Noise in Image

```
# Read grayscale image
Img_path=r"D:/DIPLab/Image/rice.tiff";
img = cv2.imread(Img_path,cv2.IMREAD_GRAYSCALE)
 # Generate Gaussian noise
 mean = 0, std = 2
noiseImage = np.random.normal(mean,std,img.shape)
print("Image Shape:", img.shape)
print("Pixel Values:")
print(img)
```

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

```
noiseValue = np.random.normal(mean,std,(3,3))
print("3×3 Gaussian Noise:")
print(noiseValue)
```

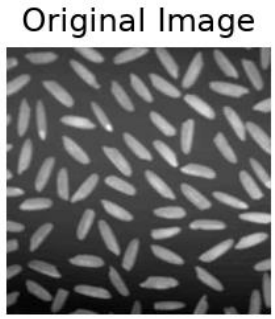
Gaussian Noise in Image

```
# Read grayscale image
Img_path=r"D:/DIPLab/Image/rice.tiff";
img = cv2.imread(Img_path,cv2.IMREAD_GRAYSCALE)


# Add noise
noisy_img = img.astype(np.float32) + noiseImage

print("Noisy Image Shape:", img.shape)


print("Pixel Values:")
print(img)
```





Gaussian Noise in Image

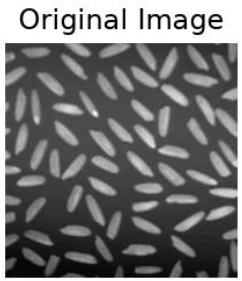
```
# Read grayscale image
Img_path=r"D:/DIPLab/Image/rice.tiff";
img = cv2.imread(Img_path,cv2.IMREAD_GRAYSCALE)
    # Select a 3×3 region from the top-left corner
    ImgPixels_3x3=cv2.resize(img,(3,3))
    print("3×3 Pixel Values:")
    print(ImgPixels_3x3)

    NoiseImageValue=ImgPixels_3x3+noiseValue
    print("3×3 Image with Noisy Value:")
    print(NoiseImageValue)

    # Keep pixel values between 0 and 255
    noisy_img = np.clip(noisy_img,0,255).astype(np.uint8)
```

Gaussian Noise in Image

```
# Display
plt.figure(figsize=(10, 4))
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(noisy_img, cmap="gray")
plt.title("Gaussian Noise Added")
plt.axis("off")
plt.show()
```

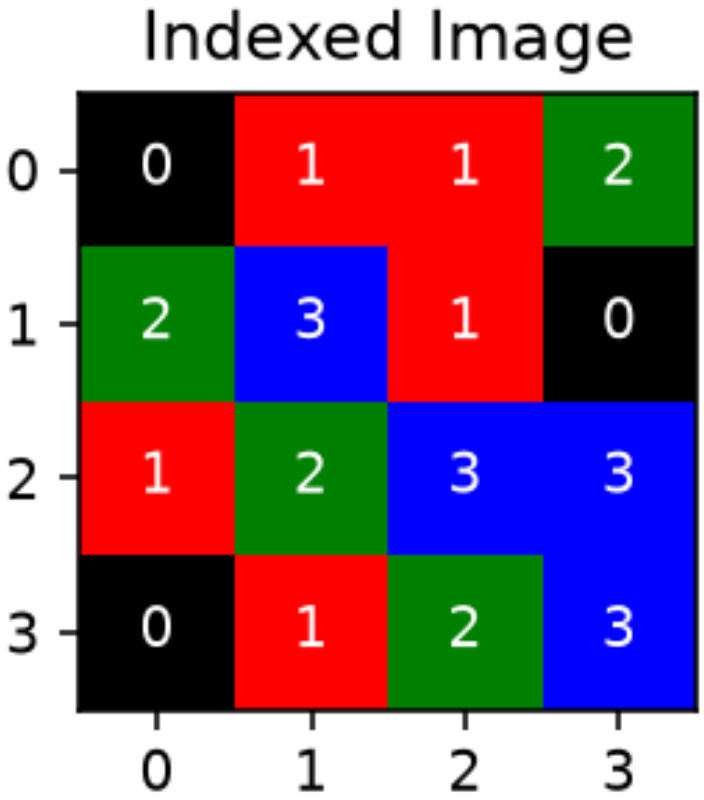




3×3 Image with Noisy Value:

```
[153.96001228  168.88797667  187.23369747]
[ 90.00280199   93.36473408   93.50453713]
[128.77036655   64.83717005   63.67267689]
```

Index Image



15	12	12	12	12	15	15	15
4	4	7	7	3	10	10	10
4	4	7	7	3	10	10	10
4	4	7	7	3	10	10	10
4	4	11	11	3	0	10	10
4	4	11	11	3	10	10	10
9	9	9	9	1	1	1	1
9	9	9	9	1	1	1	1

(b) Data matrix

	R	G	B
0	255	117	255
1	231	117	240
2	79	136	255
3	210	109	25
4	255	192	0
5	51	102	255
6	0	176	80
7	216	216	216
8	183	94	21
9	218	216	214
10	51	153	51
11	0	102	255
12	255	204	102
13	68	142	255
14	45	148	57
15	0	185	83

(c) Palette matrix

Index Image

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

# Indexed image
indexed = np.array([
   [0, 1, 1, 2],
   [2, 3, 1, 0],
   [1, 2, 3, 3],
   [0, 1, 2, 3]
])
```

```
# Color palette
colors =
["black","red","green","blue"]

cmap = ListedColormap(colors)
```

```
plt.imshow(indexed,cmap=cmap,vmin=0,vmax=3)
```

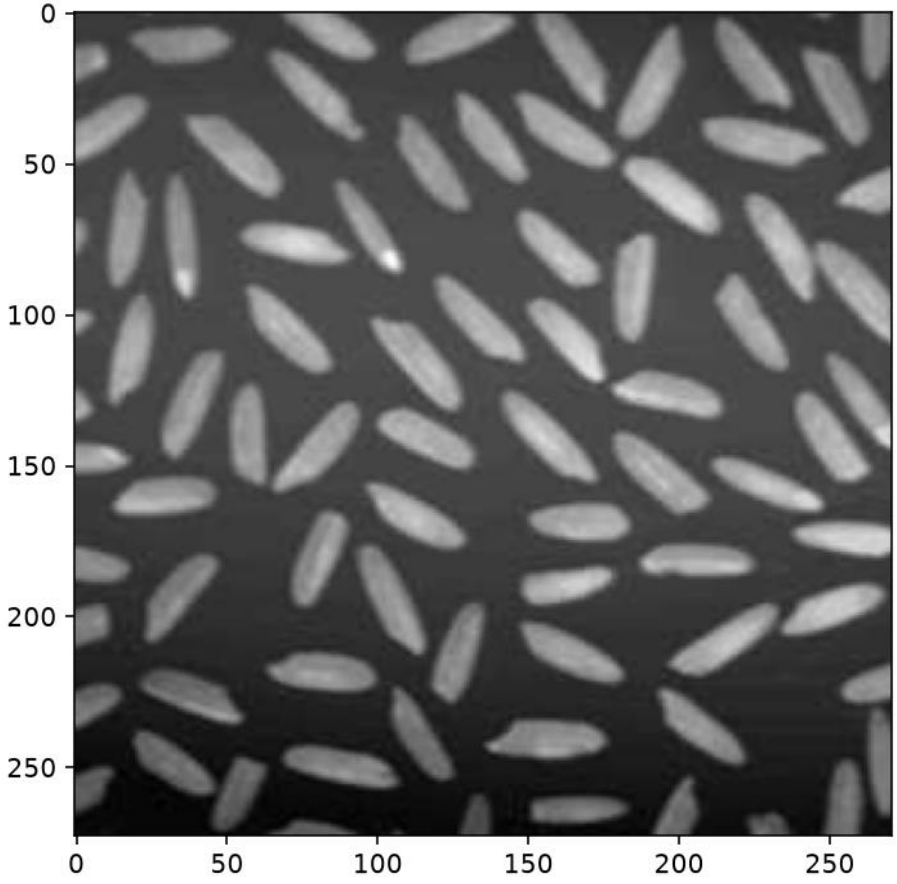
Index Image

```
palette = [
    (0, 0, 0),        # Index 0 → Black
    (255, 0, 0),      # Index 1 → Red
    (0, 255, 0),      # Index 2 → Green
    (0, 0, 255)       # Index 3 → Blue
]
```

Convert an Image into a Digital Image

Sampling + Quantization=Digital Image

Module#1.5: Image conversion to digital pixels



8 × 8 Digital Image with Pixel Values

Row \ Column	0	1	2	3	4	5	6	7
0	102	95	123	106	111	115	102	106
1	114	121	103	118	124	128	126	114
2	117	120	125	113	125	124	122	131
3	117	118	114	132	130	127	115	119
4	114	112	115	128	120	125	113	125
5	97	85	105	106	104	102	105	136
6	76	76	87	96	90	87	96	83
7	57	66	73	62	79	67	67	79

Module#1.5: Image conversion to digital pixels

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

Img_path = r"D:/DIPLab/Image/rice.tiff"

img = cv2.imread(Img_path)

# 2. Convert BGR image to Grayscale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.imshow(digital_img,cmap="gray",vmin=0,vmax=255)
```

Module#1.5: Image conversion to digital pixels

```
# ------------------------------------------------
# 3. Convert to Digital Image
#    Resize to 8 × 8 pixels
# ------------------------------------------------
digital_img = cv2.resize(gray_img,(8, 8),
      interpolation=cv2.INTER_AREA)

    # Make sure pixel values are uint8
    digital_img = digital_img.astype(np.uint8)

    print("Digital Image Pixel Values:")
    print(digital_img)
```

Module#1.5: Image conversion to digital pixels

```
# Show pixel values
    for i in range(digital_img.shape[0]):
        for j in range(digital_img.shape[1]):
            # Get ONE pixel value
            value = int(digital_img[i, j])
# Select text color
            if value > 32:
                text_color = "black"
            else:
                text_color = "white"
# Write value on image
            plt.text( j, i,
                      str(value),
                      ha="center",
                      va="center",
                      color=text_color,
                      fontsize=12,
fontweight="bold" )
```

Module#1.5: Image conversion to digital pixels

```
# 5. Plot Settings
   plt.xticks(range(8))
   plt.yticks(range(8))

   plt.xlabel("Column")
   plt.ylabel("Row")

   plt.title("8 × 8 Digital Image with Pixel Values")

  #plt.grid(True,linewidth=0.8)
   plt.tight_layout()
   plt.show()
```

Image Interpolation

Image interpolation estimates new pixel values when an image is resized, enlarged, reduced, rotated, or geometrically transformed.

Suppose:

10 20

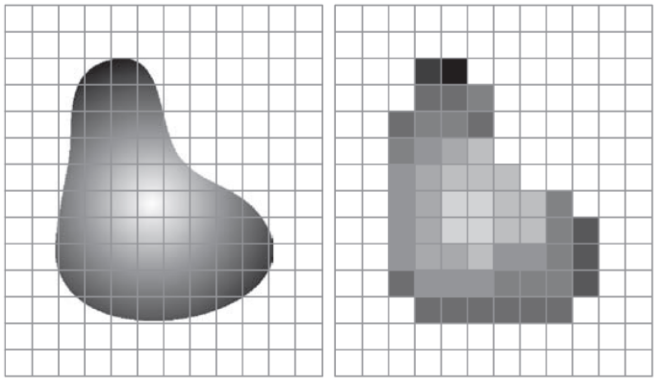
30 40

For a point between four pixels:

$$f(x,y) = (1-a)(1-b)f_0 + a(1-b)f_{10} + (1-a)bf_{01} + abf_{11}$$

where ***a*** and ***b*** describe the position between the neighboring pixels. Say, $a = 0.5, b = 0.5$

Image Sampling and Quantization



Original Image



Sampled 32×32



Sampled 64×64



Sampled 1024×1024





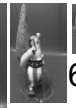
1024



512



256



128



64

32



8-bit



7-bit



6-bit



5-bit



4-bit



3-bit



2-bit



1-bit

Quantization

Image Sampling and Quantization

```
Img_path=r"D:/DIPLab/Image/Multicolor-
MagnificFlower.bmp";

img = cv2.imread(Img_path)
```

```
if img is None:
    print("Image not found!")
else:
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Image Sampling

```
plt.subplot(1,2,1)
plt.imshow(img)
plt.axis("off")
plt.show()

# ----------------------------------------
# Convert to RGB and Grayscale
# ----------------------------------------
#rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.subplot(1,2,2)
plt.imshow(gray)
```

Image Sampling

Down sampling

```
sampled1 = cv2.resize(img,(32,32),interpolation=cv2.INTER_NEAREST)

sampled2 = cv2.resize(img,(64,64),interpolation=cv2.INTER_NEAREST)

sampled3 =cv2.resize(img,(1024,1024),interpolation=cv2.INTER_NEAREST)
```

$$\begin{bmatrix} 100 & 105 & 110 & 115 & 120 & 125 \\ 130 & 135 & 140 & 145 & 150 & 155 \\ 160 & 165 & 170 & 175 & 180 & 185 \\ 190 & 195 & 200 & 205 & 210 & 215 \\ 220 & 225 & 230 & 235 & 240 & 245 \\ 250 & 255 & 240 & 225 & 210 & 195 \end{bmatrix}$$

$$Down\ Sampled\ Image = \begin{bmatrix} 100 & 110 & 120 \\ 160 & 170 & 180 \\ 220 & 230 & 240 \end{bmatrix}$$

Quantization

#labels=4;

Quantization index

#q = img * labels / 256

Convert quantization index to 8-bit intensity

#quantized = q * (255 / (labels - 1))

Quantization levels

4-level quantization

quantized1 = (img * 4 / 256) * 50

8-level quantization

quantized2 = (img * 255 / 256) * 85

plt.figure(figsize=(10, 4))

- **Quantization**

Step $= 256/8 = 32$

$$Quantized\ Image\ (3-bit) = \begin{bmatrix} 1 & 2 & 3 & 6 \\ 1 & 2 & 4 & 6 \\ 1 & 3 & 4 & 6 \\ 2 & 3 & 4 & 7 \end{bmatrix}$$

Create Pixel Connectivity Graph

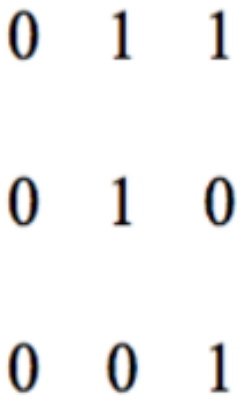


Fig: An arrangement of pixels

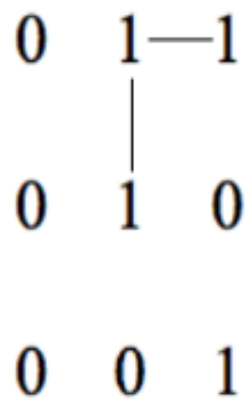


Fig: 4-connectivity of pixels

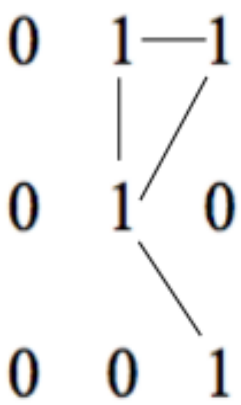


Fig: 8-connectivity of pixels

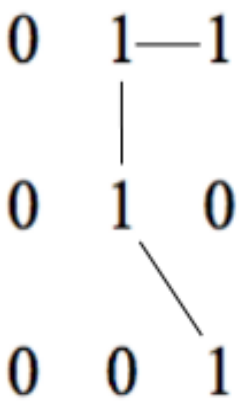


Fig: m-connectivity of pixels

```
import cv2
import networkx as nx
import matplotlib.pyplot as plt
```

```
# Example image
img = [
    [10,20,30],
    [15,25,35],
    [20,30,40]
]
```

Create Pixel Connectivity Graph

```
# Example image
img = [
    [10,20,30],
    [15,25,35],
    [20,30,40]
]
```

```
G = nx.Graph()

rows = len(img)
cols = len(img[0])
```

```
# Add nodes
for i in range(rows):
    for j in range(cols):
        G.add_node((i,j),
value=img[i][j])
```

```
# Add 4-connectivity edges
for i in range(rows):
    for j in range(cols):

        if i+1 < rows:
            G.add_edge((i,j),(i+1,j))

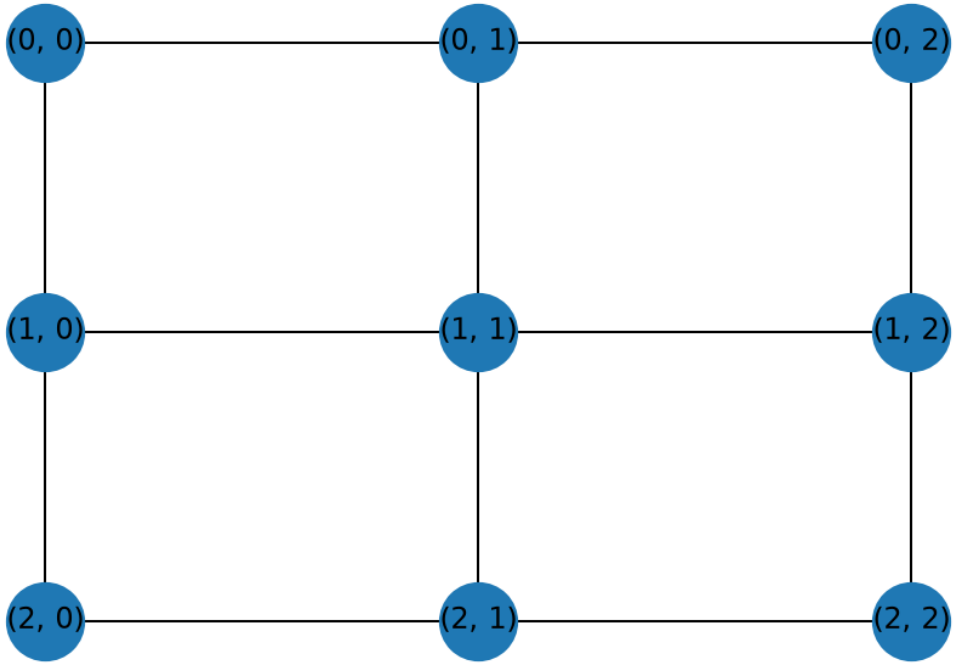
        if j+1 < cols:
            G.add_edge((i,j),(i,j+1))
```

Create Pixel Connectivity Graph

```
# Draw graph
pos = {(i,j):(j,-i) for i in range(rows) for j in range(cols)}

nx.draw(G, pos,
    with_labels=True,
    node_size=1000
)

plt.show()
```



KNN Interpolation

3. KNN Interpolation Equation

- A simple KNN interpolation can use the average of the K nearest pixels:

$$I(x,y) = \frac{1}{K}\sum_{i=1}^{K} I(x_i - y_i)$$

- A better approach is **distance-weighted KNN interpolation**, where closer pixels have greater influence. The weight is defined as

$$w_i = \frac{1}{d_i + \epsilon}$$

- where ϵ is a very small value used to avoid division by zero.
- The interpolated intensity is then

$$I(x,y) = \sum_{i=1}^{K} w_i \, I(x_i - y_i) / \sum_{i=1}^{K} w_i$$

KNN Interpolation

- Consider the original image:

$$I = [10\ 30\ 20\ 40]$$

- Suppose we want to estimate the center pixel:

$$P = (0.5, 0.5)$$

- The 4 original pixels are:

$$10, 20, 30, 40$$

- All four pixels have the same distance from the center:

$$d = \sqrt{(0.5)^2 + (0.5)^2}$$

- Therefore, their weights are equal. $For\ K = 4$:

- Thus, the new center pixel has an estimated intensity of **25**.

Image Interpolation (Bilinear Interpolation)

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Original Pixel	Value
(1,1)	10
(1,2)	20
(2,1)	30
(2,2)	40

New Position	Corresponding Original Pixel	Value
(1,1)	(1,1)	10
(1,2)	(1,1.5)	15
(1,3)	(1,2)	20
(2,1)	(1.5,1)	20
(2,2)	(1.5,1.5)	25
(2,3)	(1.5,2)	30
(3,1)	(2,1)	30
(3,2)	(2,1.5)	35
(3,3)	(2,2)	40

3×3 image:

$$\begin{bmatrix} 10 & 15 & 20 \\ 20 & \mathbf{25} & 30 \\ 30 & 35 & 40 \end{bmatrix}$$



Image Interpolation (Bilinear Interpolation)

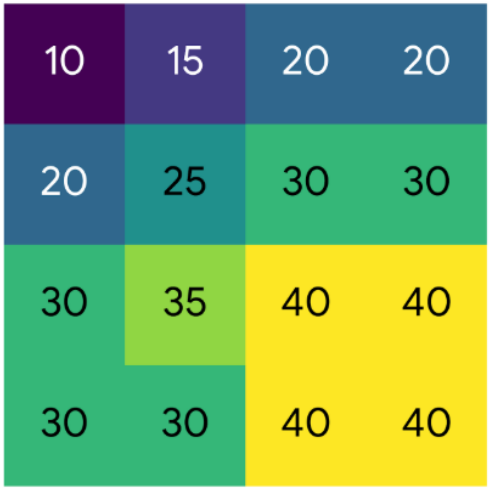
$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

New Position	Corresponding Original Pixel	Value
(1,1)	(1,1)	10
(1,2)	(1,1.5)	15
(1,3)	(1,2)	20
(1,4)	(1,2.5)	20
(2,1)	(1.5,1)	20
(2,2)	(1.5,1.5)	25
(2,3)	(1.5,2)	30
(2,4)	(1.5,2.5)	30
(3,1)	(2,1)	30
(3,2)	(2,1.5)	35
(3,3)	(2,2)	40
(3,4)	(2,2.5)	40
(4,1)	(2.5,1)	30
(4,2)	(2.5,1.5)	30
(4,3)	(2.5,2)	40
(4,4)	(2.5,2.5)	40

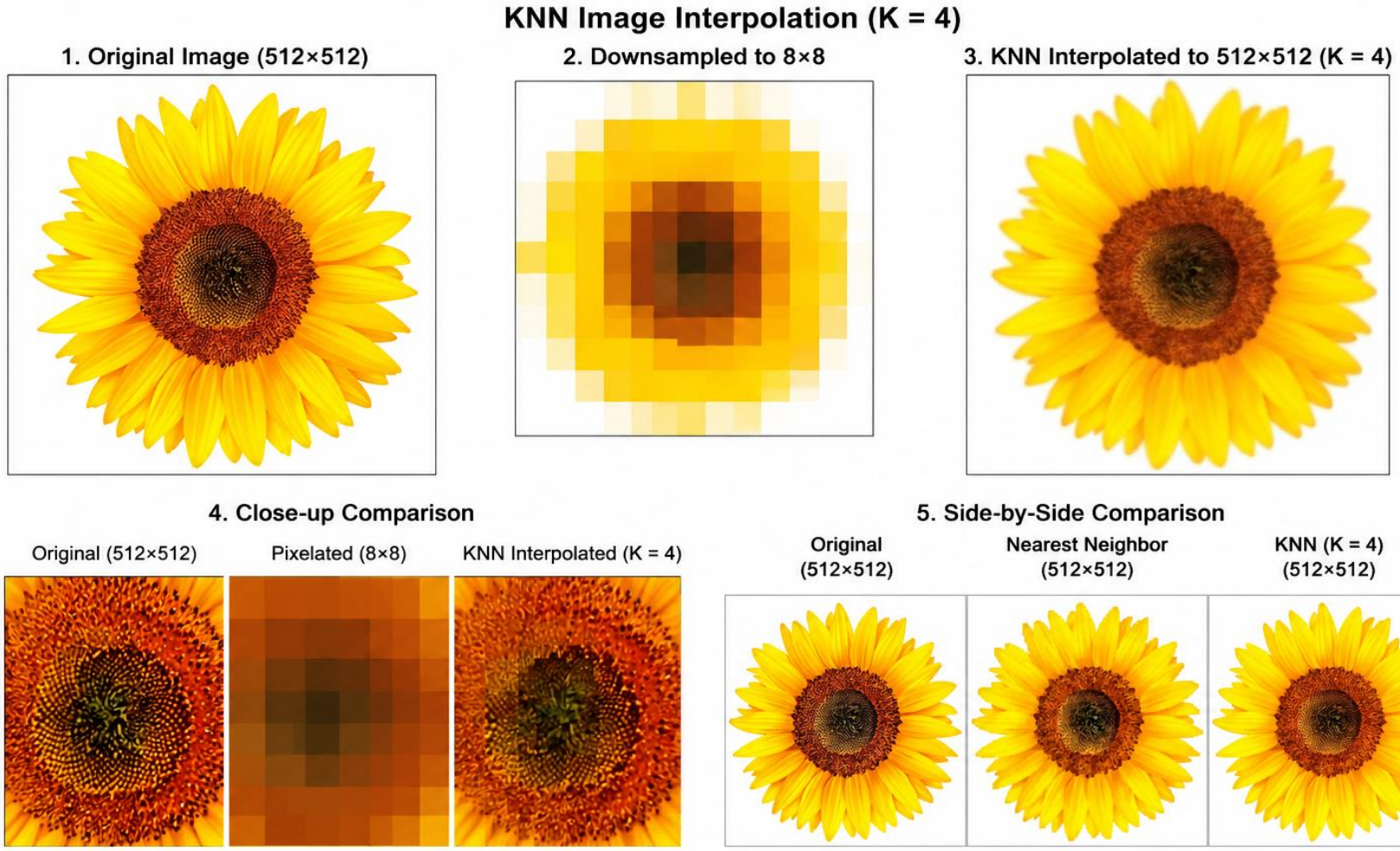
Original Pixel	Value
(1,1)	10
(1,2)	20
(2,1)	30
(2,2)	40

Resulting 4×4 image:

$$\begin{bmatrix} 10 & 15 & 20 & 20 \\ 20 & 25 & 30 & 30 \\ 30 & 35 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$



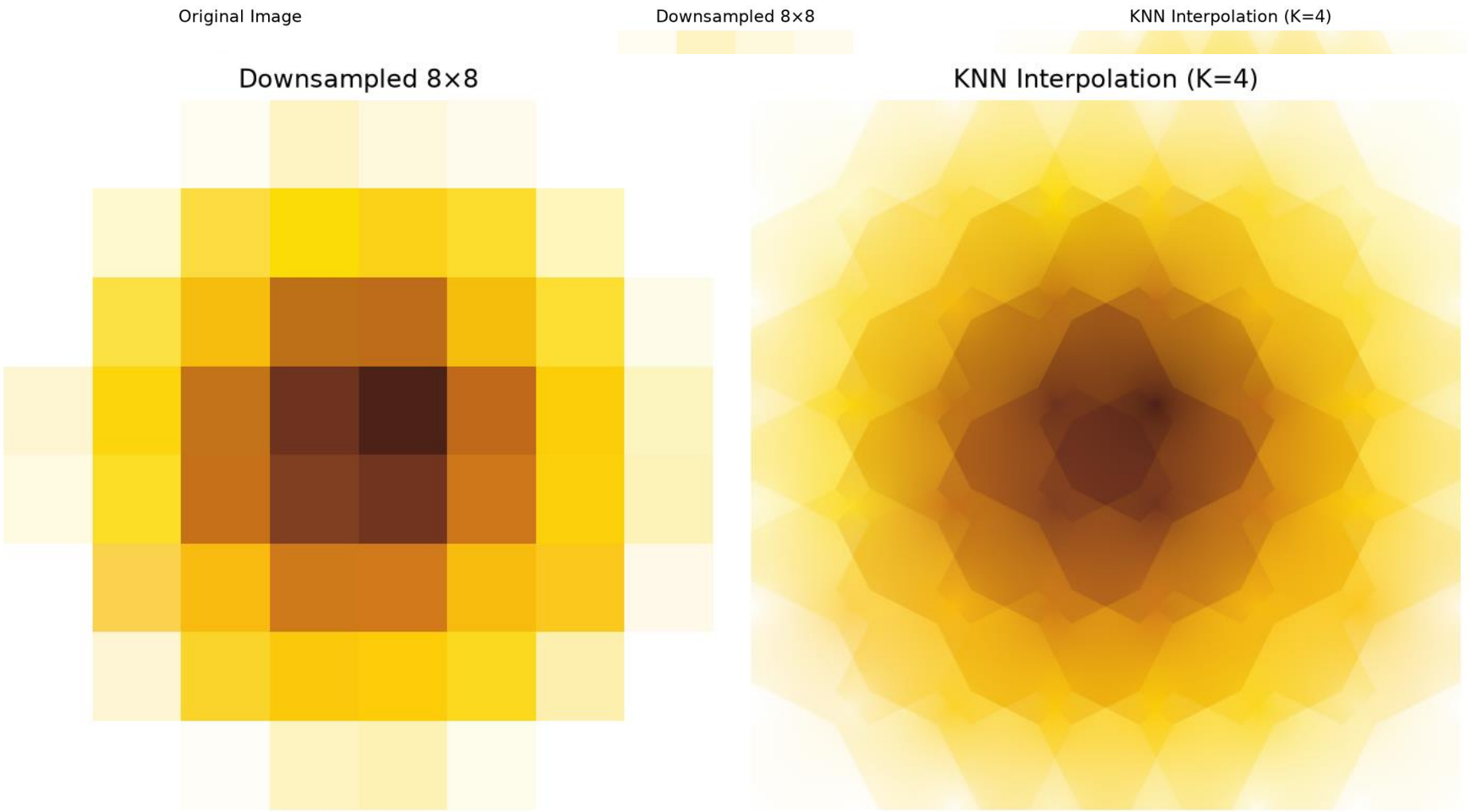
KNN Interpolation



Method : K-Nearest Neighbors Interpolation
K Value : 4 (using inverse distance weighting)
Downsample Size : 8×8
Upsample Size : 512×512

Results (vs Original)
Nearest Neighbor → PSNR: 24.31 dB, SSIM: 0.8173, MSE: 30.45
KNN (K=4) → PSNR: 29.87 dB, SSIM: 0.9286, MSE: 10.61

KNN Interpolation



Module#1.3: Color Space Conversion (RGB, BGR, Gray, HSV, LAB, YCrCb)

Color Space	Concept	What the channels represent	Why we use it
BGR	Represents a color using Blue, Green, and Red intensity values.	B = Blue, G = Green, R = Red	OpenCV's default format for reading images
RGB	Represents a color using Red, Green, and Blue intensity values.	R = Red, G = Green, B = Blue	Common for displays, cameras, and visualization
Grayscale	Represents an image using brightness/intensity only, without color information.	One channel = intensity	Simplifies images and reduces computation
HSV	Separates color (hue) from color strength (saturation) and brightness (value).	H = Hue, S = Saturation, V = Value	Useful for color detection and segmentation
LAB	Separates lightness from two color dimensions.	L = Lightness, A = Green ↔ Red, B = Blue ↔ Yellow	Useful for color analysis and operations where brightness should be separated from color

L*a*b Color Components

$$L^* = 116 \cdot h\left(\frac{Y}{Y_W}\right) - 16$$

Lightness

$$a^* = 500\left[h\left(\frac{X}{X_W}\right) - h\left(\frac{Y}{Y_W}\right)\right]$$

Red minus green

$$b^* = 200\left[h\left(\frac{Y}{Y_W}\right) - h\left(\frac{Z}{Z_W}\right)\right]$$

Green minus blue

$$h(q) = \begin{cases} \sqrt[3]{q} & q > 0.008856 \\ 7.787q + 16/116 & q \le 0.008856 \end{cases}$$

RGB → HSV, YCbCr, and Lab (via XYZ)

Given RGB Pixel

Let's take:

$$R = 100, G = 150, B = 200$$

Normalize (divide by 255):

$$R' = 0.392, G' = 0.588, B' = 0.784$$

1. RGB → HSV

Step 1: Find max, min

$$C_{max} = 0.784, C_{min} = 0.392$$

$$\Delta = C_{max} - C_{min} = 0.392$$

Step 2: Hue (H)

Since, $C_{max} = B$:

$$H = 60 \times \left(\frac{R' - G'}{\Delta} + 4\right)$$

$$= 60 \times \left(\frac{0.392 - 0.588}{0.392} + 4\right)$$

$$= 210°$$

Step 3: Saturation (S)

$$S = \frac{\Delta}{C_{max}} = \frac{0.392}{0.784} = 0.5$$

Step 4: Value (V)

$$V = C_{max} = 0.784$$

HSV Result:

$$(H, S, V) = (210°, 0.5, 0.784)$$

RGB → HSV, YCbCr, and Lab (via XYZ)

Given RGB Pixel

Let's take:

$$R = 100, G = 150, B = 200$$

Normalize (divide by 255):

$$R' = 0.392, G' = 0.588, B' = 0.784$$

YCbCr Result:

$$(Y, Cb, Cr)$$

$$\approx (\mathbf{140.75}, \mathbf{161.44}, \mathbf{98.94})$$

XYZ Result:

$$(X, Y, Z)$$

$$\approx (\mathbf{0.512}, \mathbf{0.559}, \mathbf{0.821})$$

RGB → YCbCr

Y=0.299R+0.587G+0.114B $= 140.75$

Cb=128−0.168736R−0.331264G+0.5B$= 161.44$

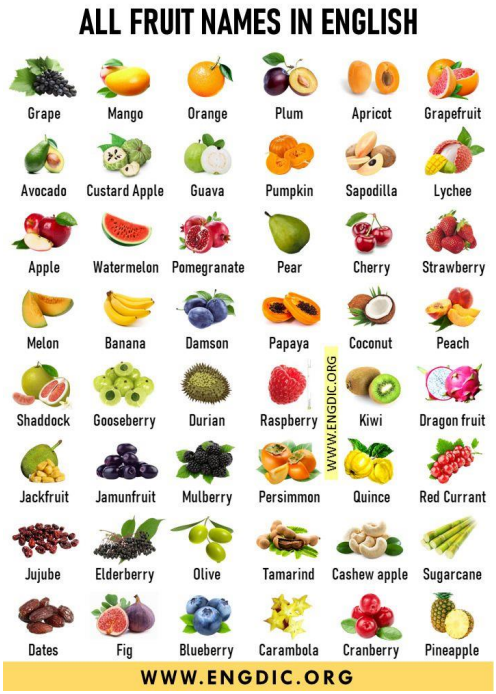
$$Cr = 128 + 0.5R - 0.418688G - 0.081312B = 98.94$$

RGB → CIE XYZ

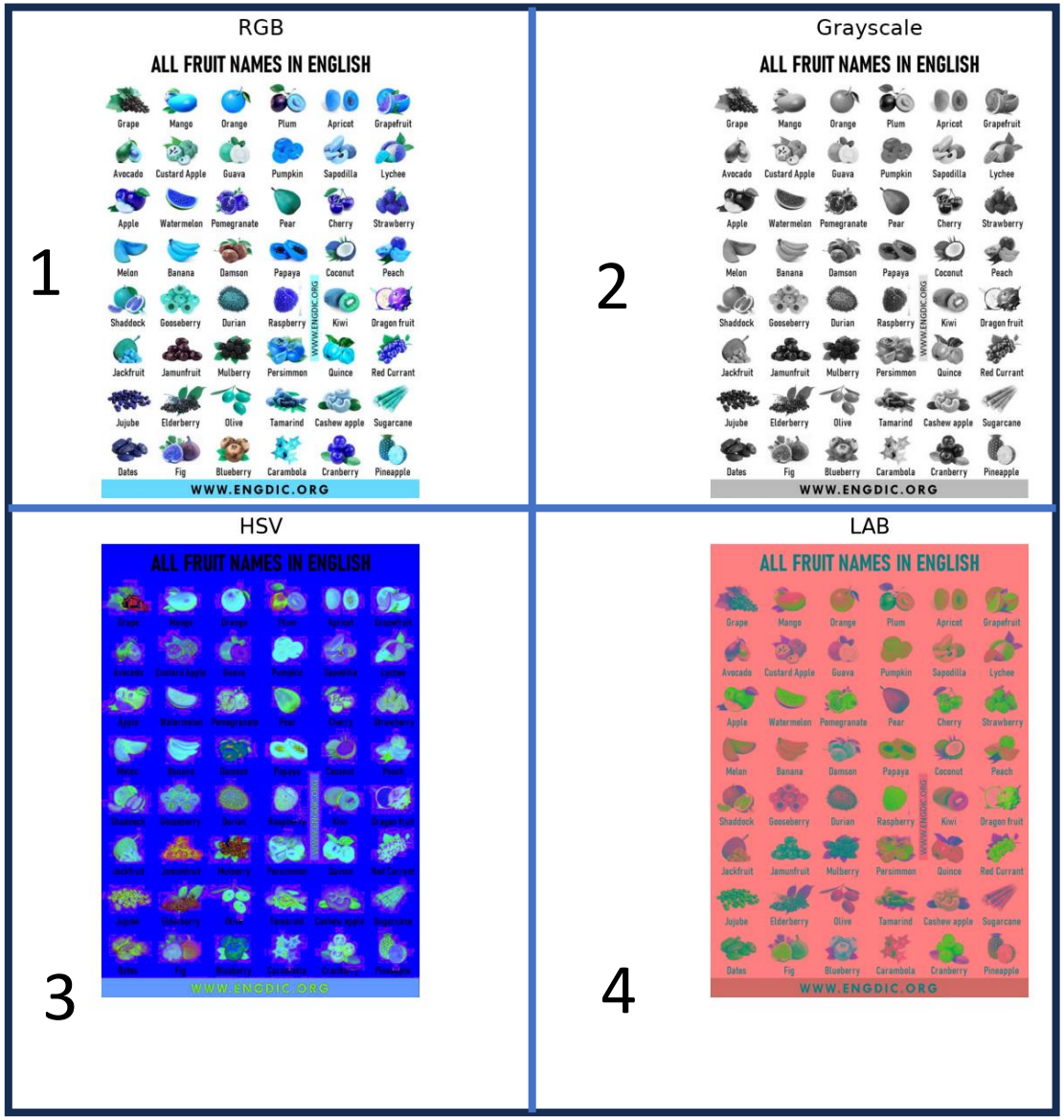
$$X = 0.412R' + 0.358G' + 0.180B' = 0.512$$
$$Y = 0.213R' + 0.715G' + 0.072B' = 0.559$$
$$Z = 0.019R' + 0.119G' + 0.950B' = 0.821$$

Module#1.3: Color Space Conversion (RGB, BGR, Gray, HSV, LAB, YCrCb)

plt.imshow(img)




```
plt.subplot(2,2,1)
plt.imshow(rgb)
plt.title("RGB")
plt.axis("off")
```



Module#1.3: Color Space Conversion (RGB, BGR, Gray, HSV, LAB, YCrCb)

```
Img_path=r"D:\DIPLab\Image\Fruits.jpg";

img = cv2.imread(Img_path)

# Color Space Conversion
# OpenCV reads images in BGR format
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
```

Module#1.3: Color Space Conversion (RGB, BGR, Gray, HSV, LAB, YCrCb)

Save Images in a folder

```
output_folder = r"D:\DIPLab\Results"
os.makedirs(output_folder, exist_ok=True)

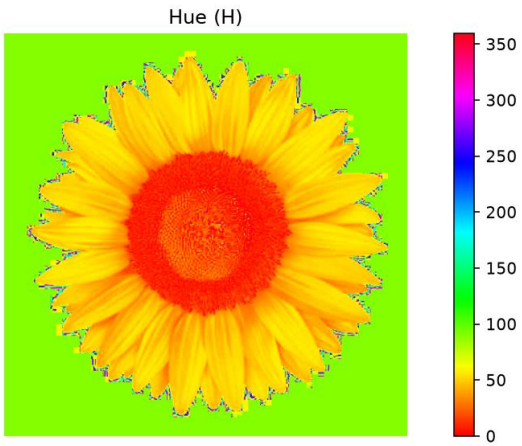
cv2.imwrite(os.path.join(output_folder, "gray.jpg"), gray)
cv2.imwrite(os.path.join(output_folder, "rgb.jpg"),

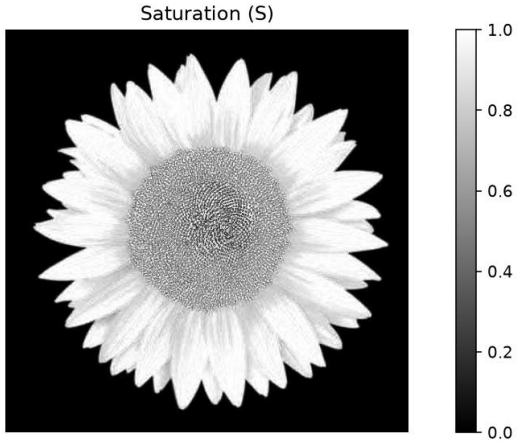
cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR))

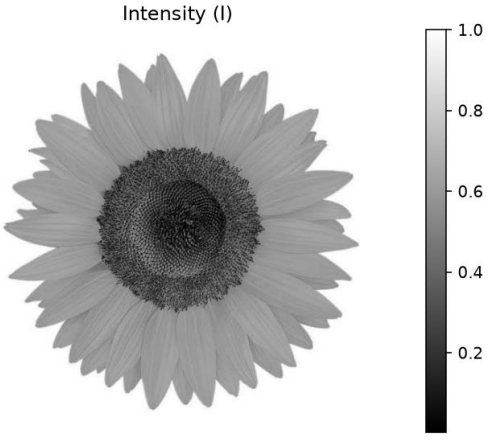
cv2.imwrite(os.path.join(output_folder, "hsv.jpg"), hsv)
cv2.imwrite(os.path.join(output_folder, "lab.jpg"), lab)
```

Example: RGB to HSI









Example: RGB to HSI

Example:

Let's say we have an RGB color value of (100, 200, 50).

1.Normalize:

R = 100/350 = 0.392,

G = 200/350 = 0.784,

B = 50/350 = 0.196

$$\theta = \cos^{-1}\left\{\frac{\frac{1}{2}\left[(R-G)+(R-B)\right]}{\left[(R-G)^2+(R-B)(G-B)\right]^{1/2}}\right\}$$

$$S = 1 - \frac{3}{(R+G+B)}\left[\min(R,G,B)\right]$$

2.Calculate H: H ≈ 100°

3.Calculate S: S ≈ 0.96

$$I = \frac{1}{3}(R+G+B)$$

4.Calculate I: I ≈ 0.505

RGB to HSI

```
import cv2
import numpy as np

image_path = r"D:\DIPLab\Image\sunflower2.jpg"

img_bgr = cv2.imread(image_path)

# Convert BGR to RGB
rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Convert RGB values from [0,255] to [0,1]
rgb = rgb.astype(np.float32) / 255.0

R = rgb[:, :, 0]
G = rgb[:, :, 1]
B = rgb[:, :, 2]
```

RGB to HSI

```
# 2. Calculate Intensity

I = (R + G + B) / 3.0

# 3. Calculate Saturation

minimum = np.minimum(np.minimum(R, G),B)

S = np.zeros_like(I)

sum_rgb = R + G + B

mask = sum_rgb != 0

S[mask] = 1 - (3 * minimum[mask] / sum_rgb[mask])
```

RGB to HSI

```
# 4. Calculate Hue

numerator = 0.5 * ((R - G) + (R - B))

denominator = np.sqrt((R - G)**2 +(R - B) * (G - B))

# Avoid division by zero
denominator = np.maximum(denominator,1e-8)

theta = np.arccos(np.clip(numerator / denominator,-1,1))

H = np.where(
   B <= G,
   theta,
   2 * np.pi - theta)
```

RGB to HSI

```
# Convert radians to degrees
H = np.degrees(H)

# 5. Handle grayscale pixels

H[~mask] = 0
S[~mask] = 0

# 6. Display RGB and HSI components
plt.figure(figsize=(15, 8))

plt.subplot(2, 2, 1)
plt.imshow(rgb)
plt.title("Original RGB Image")
plt.axis("off")
```

RGB to HSI

```
plt.subplot(2, 2, 2)
plt.imshow(H, cmap="hsv")
plt.title("Hue (H)")
plt.colorbar()
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(S, cmap="gray")
plt.title("Saturation (S)")
plt.colorbar()
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(I, cmap="gray")
plt.title("Intensity (I)")
plt.colorbar()
plt.axis("off")

plt.tight_layout()
plt.show()
```

7. Print pixel values at image center

```
height, width = R.shape

cy = height // 2
cx = width // 2

print("Center Pixel:")
print("----------------------")
print("R =", rgb[cy, cx, 0])
print("G =", rgb[cy, cx, 1])
print("B =", rgb[cy, cx, 2])
print()
print("H =", H[cy, cx], "degrees")
print("S =", S[cy, cx])
print("I =", I[cy, cx])
```

Example: HSI to RGB

Original Sunflower



HSI → RGB



Example: HSI to RGB

Input HSI Values:

Let's assume we have an HSI color with the following values: H (Hue) = **60 degrees**, S (Saturation) = 0.7, and I (Intensity) = 0.8.

RG sector $(0° \le H < 120°)$:

$$B = I(1 - S)$$

$$G = 3I - (R + B)$$

$$R = I\left[1 + \frac{S\cos H}{\cos(60° - H)}\right]$$

Result:

The RGB values for the given HSI color are:

R = 0.8, G = 0.8, and B = 0.24.

Example: HSI to RGB

Example:

Let's say you have an HSI color with **H = 120, S = 0.5, and I = 200**.

$$\textbf{\textit{GB sector}}\ (120° \le H < 240°) \qquad H = H - 120°$$

$$R = I(1 - S) \qquad G = I\left[1 + \frac{S\cos H}{\cos(60° - H)}\right] \qquad B = 3I - (R + G)$$

Example: Let's say you have **H = 150°,** S = 0.8, and I = 0.5.

$$\textbf{\textit{BR sector}}\ (240° \le H \le 360°) \qquad H = H - 240°$$

$$G = I(1 - S)$$

$$R = 3I - (G + B)$$

$$B = I\left[1 + \frac{S\cos H}{\cos(60° - H)}\right]$$

Right & Left Hand Detection

Thank You